Abstract
Rule

This report presents a comprehensive Machine Learning project aimed at predicting student academic performance (Pass/Fail) using a synthetic dataset generated via a weighted scoring function. The project encompasses the entire ML lifecycle: synthetic data generation, exploratory data analysis (EDA), feature preprocessing, model training, and evaluation. Four distinct classification algorithms—Logistic Regression, Random Forest, Support Vector Machine (SVM), and K-Nearest Neighbors (KNN)—were trained and compared. The **Random Forest** classifier emerged as the optimal model, achieving an accuracy of **86.00%** and an F1-score of **0.8627** on the test set. Additionally, an interactive web application was developed using Streamlit to deploy the model for real-time predictions.

Keywords: Machine Learning, Classification, Student Performance Prediction, Random Forest, Streamlit, Exploratory Data Analysis.

Contents
Rule

Abstract **1**

1 Introduction **3**

2 Dataset Description **3**
2.1 Feature Dictionary 3
2.2 Data Generation Logic 4

3 Methodology **4**
3.1 Preprocessing 4
3.2 Model Selection 4

4 Exploratory Data Analysis (EDA) **4**

5 Results & Evaluation **6**
5.1 Model Comparison 6
5.2 Best Model: Random Forest 6
5.2.1 Confusion Matrix 7
5.2.2 ROC Curve 7
5.2.3 Feature Importance 8

6 Web Interface Deployment **8**
6.1 Features 8
6.2 Code Architecture 8

7 Conclusion **9**

References **9**

1 Introduction
Rule

Educational institutions generate vast amounts of student data, including study habits, attendance, and socio-economic backgrounds. Leveraging this data through Machine Learning (ML) can provide early identification of students at risk of underperforming, allowing for timely interventions.

The primary objective of this project is to implement a binary classification model that predicts whether a student will **Pass** or **Fail** based on 9 distinct academic, lifestyle, and background features. To ensure a controlled and reproducible environment, a synthetic dataset of 250 samples was generated using a weighted scoring function coupled with Gaussian noise. The project demonstrates a complete end-to-end ML pipeline—from data generation to model deployment via an interactive web interface.

2 Dataset Description
Rule

Since real student data contains sensitive Personally Identifiable Information (PII), a synthetic dataset (`student_performance.csv`) was generated deterministically using a fixed random seed (42). The dataset consists of 250 records and 10 columns (9 features + 1 target).

2.1 Feature Dictionary

The features were carefully chosen to reflect realistic parameters influencing student performance:

Feature	**Type**	**Description**
`study_hours_per_day`	Float	Average daily self-study hours (0.5 – 10)
`attendance_percentage`	Float	Class attendance rate (30 – 100 %)
`previous_gpa`	Float	GPA from prior semester (0 – 10)
`sleep_hours_per_day`	Float	Average nightly sleep (3 – 10 hrs)
`extracurricular_activities`	Integer	Participation in extracurriculars (0 = No, 1 = Yes)
`internet_usage_hours`	Float	Non-academic internet use per day (0 – 8 hrs)
`parental_education_level`	Integer	Highest parental education (1=HS, 2=Bachelors, 3=Masters, 4=PhD)
`number_of_absences`	Integer	Total semester absences (0 – 30)
`assignment_completion_rate`	Float	Percentage of assignments submitted (0 – 100 %)
`performance`	**String**	**Target Variable: "Pass" or "Fail"**

Table 1: Dataset Feature Dictionary

2.2 Data Generation Logic

A latent score was calculated for each student using positive weights for constructive habits (e.g., study hours) and negative weights for detrimental ones (e.g., absences). Gaussian noise ($\mu = 0, \sigma = 2.0$) was added to ensure a fuzzy decision boundary, resulting in a realistic class balance of approximately **51.6% Pass** and **48.4% Fail**.

3 Methodology Rule

The project follows a structured pipeline executed primarily through `train_model.py` and `app.py`.

3.1 Preprocessing

- **Label Encoding:** The target variable "Pass"/"Fail" was mapped to 1 and 0 respectively using `sklearn.preprocessing.LabelEncoder`.
- **Data Splitting:** An 80/20 stratified train-test split was performed to maintain class proportions, resulting in 200 training samples and 50 testing samples.
- **Feature Scaling:** `StandardScaler` (zero mean, unit variance) was applied to distance-based and regularized models (Logistic Regression, SVM, KNN). Tree-based models (Random Forest) were trained on raw unscaled data.

3.2 Model Selection

Four diverse classifiers were chosen to provide a comprehensive comparison:

1. **Logistic Regression:** A linear baseline model.
2. **Random Forest:** An ensemble tree-based method robust to non-linearities.
3. **Support Vector Machine (RBF):** A kernel method effective in high-dimensional spaces.
4. **K-Nearest Neighbors (k=5):** An instance-based lazy learner.

4 Exploratory Data Analysis (EDA) Rule

Prior to training, EDA was conducted to understand feature distributions and correlations.

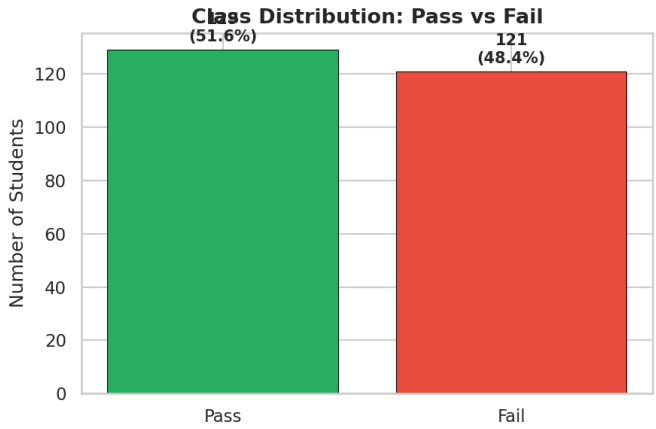

Figure 1: Class Distribution of Target Variable

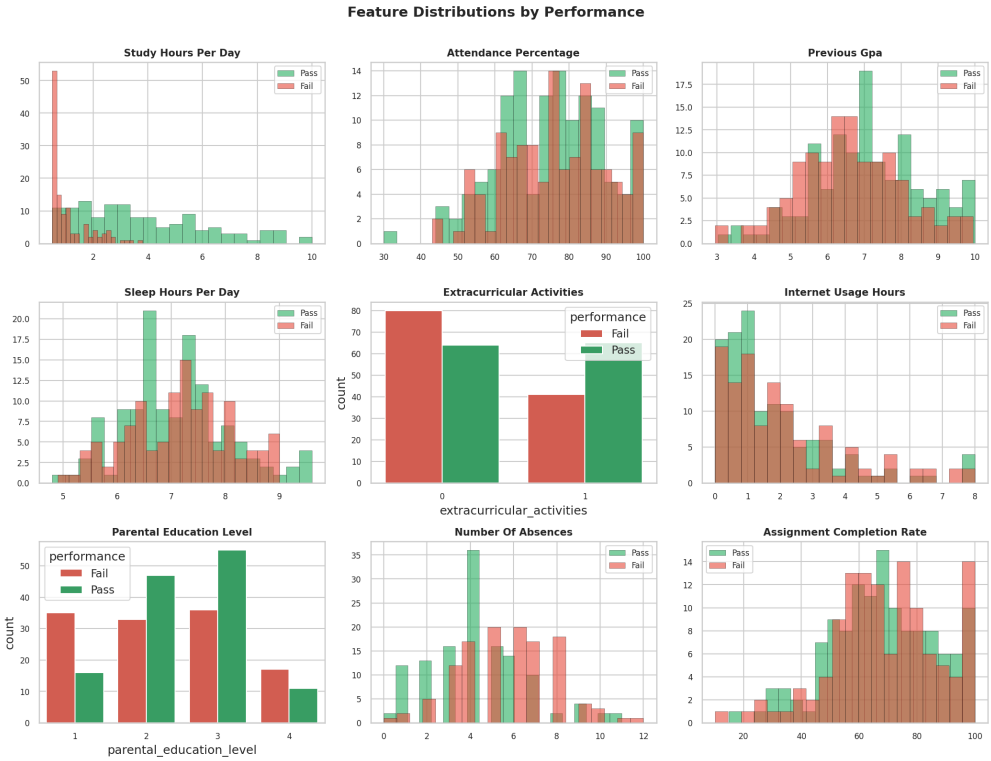

Figure 2: Feature Distributions Segmented by Pass/Fail Status

The histograms (Figure 2) reveal clear separations in certain features; for instance, students who pass tend to have higher study hours and lower absence counts compared to those who fail.

5 Results & Evaluation Rule

5.1 Model Comparison

All four models were evaluated on the hold-out test set using Accuracy, Precision, Recall, and F1-Score. Additionally, 5-Fold Cross-Validation (CV) was performed on the training data to check for overfitting.

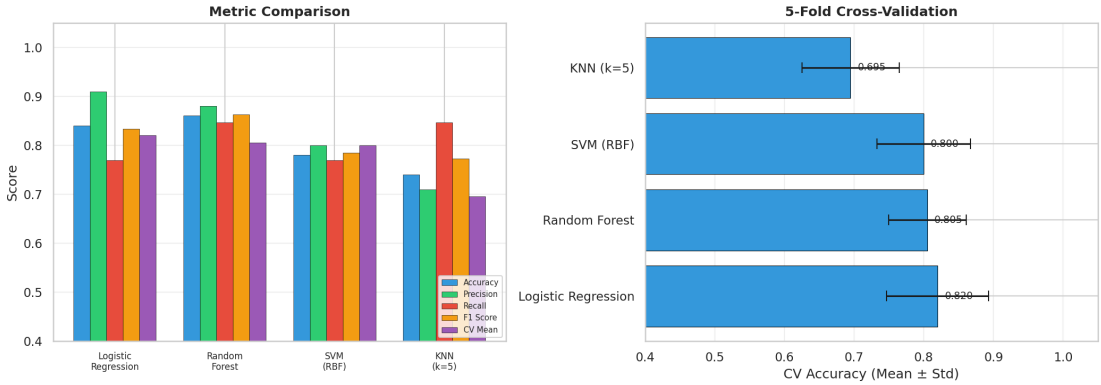

Figure 3: Metric Comparison and 5-Fold Cross-Validation Results

Model	Accuracy	Precision	Recall	F1 Score	CV Mean	CV Std
Logistic Regression	0.84	0.909	0.769	0.833	0.820	0.073
Random Forest	**0.86**	**0.880**	**0.846**	**0.863**	0.805	0.056
SVM (RBF)	0.78	0.800	0.769	0.784	0.800	0.067
KNN (k=5)	0.74	0.710	0.846	0.772	0.695	0.070

Table 2: Quantitative Model Comparison

5.2 Best Model: Random Forest

Random Forest achieved the highest F1-Score (0.8627), balancing precision and recall effectively. Its low standard deviation in cross-validation (0.0557) indicates high stability.

5.2.1 Confusion Matrix

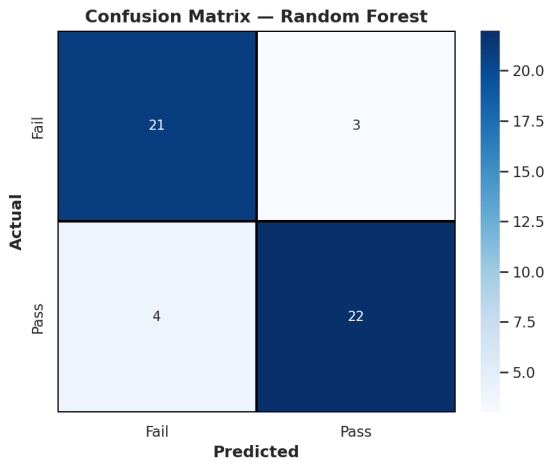

Figure 4: Confusion Matrix for Random Forest

Out of 50 test samples, the model misclassified only 7 students (3 False Positives, 4 False Negatives).

5.2.2 ROC Curve

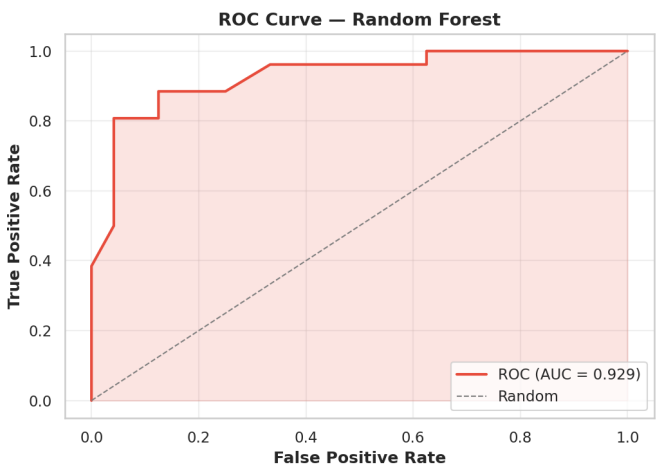

Figure 5: Receiver Operating Characteristic (ROC) Curve

5.2.3 Feature Importance

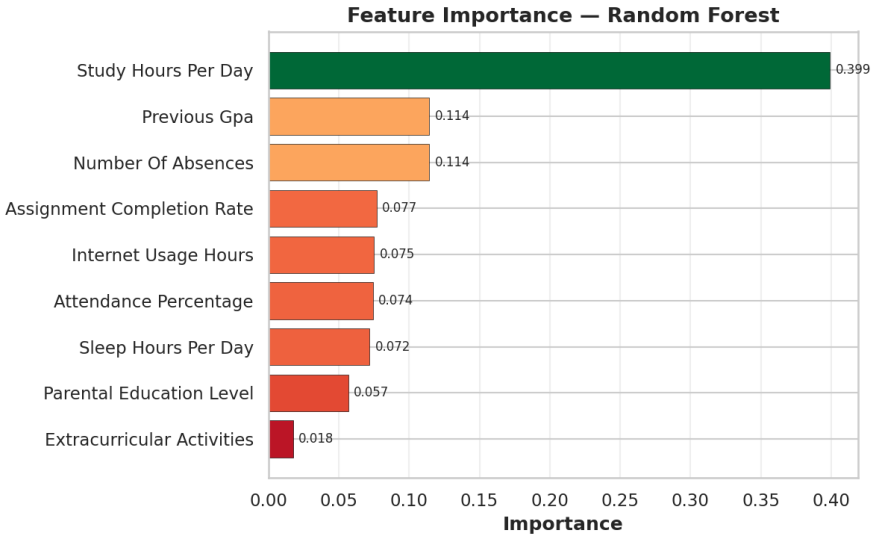

Figure 6: Feature Importance extracted from Random Forest

`study_hours_per_day` is overwhelmingly the most critical predictor (importance = 0.399), followed by `previous_gpa` and `number_of_absences`.

6 Web Interface Deployment Rule

To make the model accessible, an interactive web application was developed using **Streamlit** (`app.py`).

6.1 Features

- **Sidebar Input Form:** Categorized sliders and dropdowns (Academic, Lifestyle, Background) to input the 9 features.
- **Real-time Preprocessing:** The app applies the identical `StandardScaler` saved during training.
- **Visual Output:** Displays Pass/Fail status with confidence percentages, progress bars, and a dynamically generated feature importance chart.
- **Model Info Dashboard:** Landing page displays the selected model's metrics and an explanation of the pipeline.

6.2 Code Architecture

The app utilizes Streamlit's `@st.cache_resource` decorator to load the serialized `model_artifacts.pkl` (containing the model, scaler, and feature metadata) only once, ensuring fast subsequent predictions. Custom HTML/CSS injects styled result boxes (green for Pass, red for Fail).

7 Conclusion
Rule

This project successfully demonstrates an end-to-end Machine Learning pipeline for educational data mining. By generating a controlled synthetic dataset, we circumvented data privacy issues while maintaining realistic feature-target relationships. The comparative analysis validated that ensemble methods like Random Forest outperform linear models for this specific non-linear feature space. The deployment via Streamlit bridges the gap between backend ML scripts and end-user utility. Future work could involve integrating real institutional data and expanding the prediction to multi-class classification (e.g., Grades A, B, C, D, F).

References
Rule

1. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *JMLR*, 12, 2825–2830.

2. Streamlit Inc. (2023). Streamlit: The fastest way to build and share data apps. `https://streamlit.io`

3. McKinney, W. (2010). Data Structures for Statistical Computing in Python. *Proceedings of the 9th Python in Science Conference*, 56–61.

4. Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering*, 9(3), 90–95.